

CSCE 633 MACHINE LEARNING · MODULE 09 · LECTURE NOTES

Dimensionality Reduction

Fewer numbers, and what gets lost.

MODULE OUTCOMES

- Derive PCA and explain its relationship to the SVD.
- Choose a component count honestly.
- Explain the manifold assumption.
- Use t-SNE and UMAP, and read their output correctly.
- Explain why reduction can hurt supervised performance.

1 Principal component analysis

$\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^T \rightarrow$ the top k columns of $\mathbf{V}$ are the principal directions

Three equivalent definitions. PCA finds the successive orthogonal directions of maximum variance; equivalently, it finds the k -dimensional linear subspace minimising squared reconstruction error; equivalently, it takes the top k right singular vectors of the centred data matrix. **That maximising variance and minimising reconstruction error turn out to be the same optimisation is worth noticing** — the two intuitions feel different and the algebra identifies them.

CENTRE, USUALLY SCALE, AND COMPUTE IT WITH THE SVD

Centring is mandatory. PCA on uncentred data spends its first component on the direction from the origin to the data's mean, which carries no information about the structure and displaces everything else.

Scaling is usually necessary too. PCA maximises variance, and variance has units — so a feature measured in millimetres has a million times the variance of the same quantity in metres and will dominate every component. **Without standardisation the result reflects your choice of units rather than your data**, which is rarely what is intended. (The exception is when all features are genuinely in the same units and their relative magnitudes are meaningful, such as pixel intensities.)

Compute it from the SVD of $\mathbf{X}$, never from the eigendecomposition of $\mathbf{X}^T\mathbf{X}$ — exactly the conditioning argument of Module 02 §1, and every library does the right thing if you let it.

And fit it inside the cross-validation fold (Module 04 §2). **PCA learns from the data**, so fitting it on the full dataset before splitting leaks the test set's covariance structure into training — a smaller effect than feature selection leakage, and a real one.

CHOOSING THE NUMBER OF COMPONENTS, HONESTLY

'Retain 95% of the variance' is a convention, not a criterion. Depending on the eigenvalue spectrum it can mean two components or two hundred, and the threshold has no justification beyond familiarity.

The scree plot shows where the eigenvalues flatten, which asks you to find a kink in a curve — the same judgement, with the same unreliability, as k-means' elbow (Module 08 §1).

If the PCA output feeds a supervised model, tune the component count by cross-validation like any other hyperparameter. That is the only honest criterion available, because it measures the thing you actually care about.

And low variance does not mean low usefulness. A direction along which the data barely varies can be precisely the direction separating two classes — and **PCA will discard it, because PCA is unsupervised and never looks at y at all** (§4).

2 The manifold hypothesis

REAL DATA LIES NEAR A LOW-DIMENSIONAL SURFACE

High-dimensional data that comes from the real world occupies a tiny, highly structured subset of its ambient space. A one-megapixel grayscale image lives in a million-dimensional space, and **the set of images that look like photographs is a vanishingly small curved region of it** — almost every point in that space is noise.

So the *intrinsic* dimension is far below the ambient dimension, and most of the coordinates are redundancy rather than information. That is why compression works, why models with far fewer parameters than pixels can succeed, and why reduction is possible at all.

PCA finds a linear subspace. If the true manifold is curved — and it generally is — then a linear approximation needs many more dimensions than the manifold intrinsically has, because it must contain the curve's bounding flat.

Nonlinear methods — t-SNE, UMAP, autoencoders (CSCE 636) — attempt to follow the curvature. Which is why their output looks so much better than PCA's, and why it is so much easier to misread: a method that distorts distances in order to flatten a curve produces a picture whose distances cannot be trusted (§3).

3 t-SNE and UMAP

What you see in the plot	What it means
Two points close together	They were probably close in the original space. This is trustworthy — local neighbourhood structure is what these methods optimise for.
The distance between two clusters	Meaningless. Global distances are not preserved and the optimisation has no term encouraging them to be. Two clusters rendered far apart may be adjacent.
The relative sizes of clusters	Meaningless. t-SNE deliberately equalises density, so a tight cluster of a thousand points and a diffuse cluster of fifty can appear the same size.
The number of clusters	Depends strongly on perplexity (t-SNE) or n_neighbors (UMAP), and changes substantially across reasonable settings.
Apparent clusters in data that has none	t-SNE produces these from pure noise. This is well documented and easy to reproduce, and it means a clustered-looking plot is not evidence of clusters.
Differences between runs	Expected — the optimisation is stochastic and non-convex, so the embedding differs each time.

USE THEM TO LOOK, NOT TO CONCLUDE

Run at several perplexity or neighbour values and keep only what is stable across all of them.

Structure that appears at one setting and vanishes at another is an artifact of the setting.

Colour the embedding by something you already know — the label, the batch, the collection date, the instrument — and ask whether the structure corresponds to it. **Discovering that the embedding separates by *batch* rather than by *biology* is an extremely common and extremely valuable finding,** and it is one of the best uses of these methods.

Never run a clustering algorithm on the embedding. The embedding distorted the distances on purpose; clustering those distances compounds two sets of assumptions. **Cluster in the original space or on PCA output, and use the embedding only to *display* the result.**

UMAP preserves somewhat more global structure than t-SNE and is considerably faster, which makes it the better default. **The caveats above still apply** — less severely, not differently — and the authors of UMAP say so themselves in their documentation, which is to their credit.

4 When not to reduce

REDUCTION BEFORE A SUPERVISED MODEL OFTEN HURTS

PCA discards the directions of lowest variance, and it chooses them without ever examining the labels. The criterion is 'how much does the data spread here', not 'how much does the outcome depend on this'.

So a low-variance direction that perfectly separates the classes is discarded. This is not a hypothetical: it is easy to construct two Gaussians separated along a direction with small spread and large variance along an irrelevant axis, and PCA will throw away exactly the useful coordinate. **It also happens in practice,** particularly when a few strong nuisance factors dominate the variance.

And modern models handle many features perfectly well. Regularisation (Module 04) shrinks the useless ones and tree ensembles (Module 06) ignore them, so the historical motivation — too many features for the model to cope with — is much weaker than it was.

So reduce for a stated reason: speed, memory, visualisation, or an explicit argument that the discarded directions are noise. Not reflexively, as a preprocessing habit. And measure with and without — it is one additional cross-validation run and it settles the question for your data rather than in general.

- **Linear discriminant analysis.** Finds the directions that best separate known classes — **supervised, so it keeps exactly what PCA discards.** Limited to at most $(\text{classes} - 1)$ dimensions, which is restrictive for few classes.
- **Feature selection** (Module 04 §1). Keeps original features rather than linear combinations, **so the result stays interpretable** — a real advantage when the output must be explained.
- **Autoencoders.** Nonlinear reduction learned by reconstruction, with the bottleneck width as the dimension (CSCE 636). More capable and far more to tune.
- **Random projection.** The Johnson–Lindenstrauss lemma says that projecting onto $O(\log n / \epsilon^2)$ random directions preserves all pairwise distances to within ϵ , *regardless of the original dimension*. **No fitting, therefore no leakage risk, a provable distortion bound, and it is nearly free — it deserves considerably more attention than it receives.**
- **Or no reduction at all,** which is frequently the right answer and should be the baseline the others are measured against.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Hastie et al. — The Elements of Statistical Learning, chapter 14 (free PDF)	PCA with all three derivations of §1, and the manifold methods.

Resource	What it covers for this module
Wattenberg, Viégas & Johnson — How to Use t-SNE Effectively (free, interactive)	The §3 table, made interactive. Work through it — seeing t-SNE produce clusters from noise yourself is what makes the caveats stick.
McInnes, Healy & Melville — UMAP (free), and the documentation	The method, and an unusually honest section on what the plots do not show.
Johnson & Lindenstrauss; and scikit-learn's random projection module (free)	The §4 alternative, with the distortion bound and the dimension estimate.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement PCA via the SVD and verify it against the library.
- 2 **Run PCA without centring** and show the first component points at the mean.
- 3 **Run it on unscaled data** where one feature is in millimetres, and show that feature dominates.
- 4 **Construct data where the class-separating direction has low variance**, and show PCA discards it while LDA keeps it.
- 5 Tune the component count by cross-validation on a supervised task and compare against the 95%-variance convention.
- 6 **Run t-SNE on uniform random noise** and observe the clusters it produces.
- 7 **Vary perplexity over a wide range** and report which structures survive.
- 8 Colour a t-SNE embedding by a known batch variable and look for batch effects.
- 9 Compare clustering in the original space against clustering on the embedding, and explain the difference.
- 10 **Compare random projection against PCA** on a supervised task at equal output dimension.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give three equivalent definitions of PCA.
- 2 Why is centring mandatory and scaling usually necessary?
- 3 How should the component count be chosen?
- 4 Why can PCA discard the most useful direction?
- 5 State the manifold hypothesis and say what it explains.
- 6 In a t-SNE plot, what is meaningful and what is not?

- 7 Why should you not cluster on a t-SNE embedding?
- 8 Give four reasons to reduce and four alternatives.